Oracle DBA: Database Slow?

5 Queries to Run First

When the database is slow, don't guess. Run a few simple queries to find the real issue — fast!

1 Check Wait Events

Find out where the time is going.

```
SELECT wait_class,
       ROUND(time_waited/100,2)
AS time_sec
FROM   v$system_wait_class
WHERE  wait_class <> 'Idle'
ORDER BY time_waited DESC;
```

Look for:

- CPU
- User I/O
- System I/O
- Concurrency
- Commit

2 Check Top SQL by CPU

Identify CPU-heavy SQL.

```
SELECT sql_id, executions,

ROUND(cpu_time/1000000,2) AS
cpu_sec,

ROUND(elapsed_time/1000000,2)
AS elapsed_sec
FROM   v$sql
WHERE  cpu_time > 0
ORDER BY cpu_time DESC
FETCH FIRST 10 ROWS ONLY;
```

Look for:

- High CPU time
- Many executions
- Complex queries or bad plans

3 Check Top SQL by I/O

Find queries causing heavy reads.

```
SELECT sql_id, executions,

ROUND(physical_read_bytes/1024/1024/1024,2)
        AS physical_read_gb
FROM   v$sql
WHERE  physical_read_bytes > 0
ORDER BY physical_read_bytes DESC
FETCH FIRST 10 ROWS ONLY;
```

Look for:

- Full table scans
- Missing indexes
- Inefficient predicates

4 Check the Execution Plan

Understand why the SQL is slow.

```
SELECT *
FROM
TABLE(DBMS_XPLAN.DISPLAY_CURSOR(
  'YOUR_SQL_ID',
  NULL,
  'ALLSTATS LAST'
));
```

Look for:

- E-Rows vs A-Rows
- Full table scans
- High buffer activity
- Unexpected joins

5 Check Active Sessions

See what's currently happening.

```
SELECT sid, serial#, username,
status,
       sql_id, event
FROM   v$session
WHERE  status = 'ACTIVE'
ORDER BY seconds_in_wait DESC;
```

Look for:

- Long waits
- Lock waits
- Blocking sessions

Quick Troubleshooting Flow

1. Check wait events (where is time going?)
2. Find top SQL by CPU / I/O
3. Check execution plan (why is it slow?)
4. Look at active sessions (current blockers?)
5. Tune the SQL or fix the issue

DBA Tip

Don't assume the problem is CPU, I/O or a single SQL.

Use data to find the real cause — then fix it.

Better insights → Faster diagnosis → A healthier database!

ORACLE DBA | MONITOR • TROUBLESHOOT • OPTIMIZE

Oracle DBA: How to Resolve the Common Issues

Use the evidence from the first 5 checks before making changes.

Issue	What to Confirm	Resolution / Action
1. High CPU	Confirm the top SQL and execution frequency first. Check whether the plan changed or the SQL is executing excessively.	• Review execution plan and statistics. • Tune predicates / joins and add or adjust indexes where justified. • Check for plan regression and use SQL Plan Management when appropriate. • If workload genuinely increased, review CPU sizing after SQL tuning.
2. High User I/O / Physical Reads	Identify the SQL generating the reads. Do not increase I/O capacity before finding the workload causing it.	• Check for full table scans and inefficient access paths. • Verify indexes and optimizer statistics. • Reduce unnecessary columns/rows and improve predicates. • For unavoidable workload, review storage/IOPS capacity and cache effectiveness.
3. Concurrency / Lock Waits	Find the blocking session and the SQL holding the lock. Determine why the transaction is open.	• Identify blocker and blocked sessions. • Ask the application owner to commit/rollback the long transaction. • Fix application transaction scope if locks are held too long. • Kill a session only after confirming business impact and approval.
4. Commit / Log File Sync	Frequent commits or slow redo/log I/O can increase commit latency.	• Review commit frequency in the application. • Avoid unnecessary commit-per-row patterns. • Check redo generation and log/IO performance. • Investigate storage latency before changing database parameters.
5. Bad Execution Plan	Compare the current plan with a known good plan and check whether statistics, data distribution, or bind values changed.	• Gather appropriate optimizer statistics. • Check stale/missing statistics and histograms where justified. • Investigate bind-peeking / adaptive cursor behavior. • Consider SQL Plan Management for a proven stable plan.
6. Blocking Sessions	Determine the blocker, object, transaction age, and business impact before taking action.	• Use V$SESSION / V$LOCK / V$SQL to identify the chain. • Contact the owning application/team. • Commit or rollback the blocking transaction when appropriate. • Use ALTER SYSTEM KILL SESSION only as a

		controlled last resort.
7. Full Table Scan	A full scan is not automatically a problem. Check table size, selectivity, predicates, and actual rows returned.	• If highly selective, consider a suitable index. • Check whether functions on columns prevent index usage. • Verify optimizer statistics. • For large reporting queries returning most rows, a full scan may be correct.
8. Stale Optimizer Statistics	Stale statistics can cause poor cardinality estimates and bad plans.	• Check DBA_TAB_STATISTICS for stale statistics. • Gather statistics during an appropriate maintenance window. • Avoid blindly gathering statistics on every table during peak workload. • Compare the execution plan before and after the change.

Safe DBA Resolution Sequence

✓ Measure → identify the dominant wait / workload.

✓ Trace → identify the SQL, session, object, and execution plan.

✓ Validate → confirm the root cause with actual evidence.

✓ Fix → tune SQL, statistics, indexing, transaction behavior, or capacity as appropriate.

✓ Verify → compare waits, elapsed time, CPU, I/O and application response after the change.

✓ Document → record the root cause, action, result and rollback plan.

Important: Do not kill sessions, add indexes, gather statistics, or resize the database based on symptoms alone.

Ruben Raj

REAL-TIME ORACLE DATABASE ANALYSIS

Live troubleshooting checklist — capture evidence first, then resolve the root cause.

1. Current Active Sessions

Find active sessions, waits and possible blockers.

```
SELECT sid, serial#, username, status,
       sql_id, event, wait_class,
       seconds_in_wait, blocking_session
FROM v$session
WHERE status = 'ACTIVE'
ORDER BY seconds_in_wait DESC;
```

2. Top SQL — CPU

Identify SQL consuming the most CPU.

```
SELECT sql_id, executions,
       ROUND(cpu_time/1000000,2) cpu_sec,
       ROUND(elapsed_time/1000000,2) elapsed_sec,
       buffer_gets, disk_reads
FROM v$sql
ORDER BY cpu_time DESC
FETCH FIRST 10 ROWS ONLY;
```

3. Top SQL — Physical I/O

Identify SQL generating heavy physical reads.

```
SELECT sql_id, executions,
       ROUND(physical_read_bytes/1024/1024/1024,2)
       physical_read_gb,
       disk_reads, buffer_gets
FROM v$sql
ORDER BY physical_read_bytes DESC
FETCH FIRST 10 ROWS ONLY;
```

4. Current Wait Events

Find the dominant wait events affecting the instance.

```
SELECT event, wait_class,
       total_waits,
       ROUND(time_waited/100,2) time_sec
FROM v$system_event
ORDER BY time_waited DESC
FETCH FIRST 15 ROWS ONLY;
```

REAL-TIME ANALYSIS → ROOT CAUSE → ACTION

WAIT: What is the database waiting for?

SQL: Which SQL/session is consuming the resource?

PLAN: Why is the SQL taking that path?

OBJECT: Which table/index/object is involved?

ACTION: Tune SQL, fix statistics/indexing, resolve blocking, or address capacity.

VERIFY: Re-run the same checks and confirm improvement.

Oracle note: V$SESSION provides current session/wait information, while V$SYSTEM_EVENT provides instance-level wait statistics. Real-Time SQL Monitoring can provide near-real-time execution metrics for monitored SQL.